The models of Oka (2026), fitted with inlavanilla

One data set from the hardest condition within reach, and the recovery of its parameters

2026-10-15

Motonori Oka's thesis (Oka, 2026) estimates large latent variable models by marginal maximum likelihood: stochastic optimisation, with Langevin samplers for the latent variables. `inla_vanilla()` takes any log-posterior of the parameters, so each model here integrates its latent variables out inside `logpost`, by a quadrature rule built for that model, and hands the log-posterior of the parameters alone to `inla_vanilla()`. The code is in `lab/oka-thesis-models.R`, and `lab/oka-thesis.R` runs further checks.

Each section takes the hardest condition of the thesis's simulation study that this route can reach, simulates one data set from it with a fixed seed, fits it, and checks the recovery of every parameter in three ways:

1. the true value lies in its 95% credible interval;
2. its standardised error, $z_j = (\hat{\vartheta}_j - \vartheta_j^*)/\mathrm{sd}_j$, with $\hat{\vartheta}_j$ the posterior mean, is small, and the z_j look like draws from $N(0,1)$;
3. jointly, the squared Mahalanobis distance of the truth from the posterior, $d^2 = (\hat{\vartheta}-\vartheta^*)^\top \hat{\Sigma}^{-1}(\hat{\vartheta}-\vartheta^*)$, with $\hat{\Sigma}$ the posterior covariance from 4,000 draws, is not extreme against χ^2_m.

Each fit reports its time, stage by stage. The times are wall-clock seconds for one R process, on arm64 with R version 4.6.1 (2026-06-24), and the summary at the end collects them.

With weak priors and this much data, the Bernstein–von Mises theorem makes the posterior close to the sampling distribution of the estimate, so over repeated data sets z_j is roughly $N(0,1)$ and d^2 roughly χ^2_m. One data set cannot prove more than that the fit is consistent with the truth, but a wrong fit would fail these checks.

```
# The recovery table, the joint check and the plot of the z_j
recovery_check <- function(fit, truth, nsamp = 4000) {
  draws <- sampling(fit, nsamp = nsamp)
  post_mean <- colMeans(draws)
  post_sd <- apply(draws, 2, sd)
  q <- apply(draws, 2, quantile, c(0.025, 0.975))
  tab <- data.frame(
    true = truth,
    mean = post_mean,
    sd = post_sd,
```

```
    lower = q[1, ],
    upper = q[2, ],
    z = (post_mean - truth) / post_sd,
    covers = q[1, ] <= truth & truth <= q[2, ]
  )
  d2 <- mahalanobis(truth, post_mean, cov(draws))
  list(
    table = tab,
    d2 = d2,
    m = length(truth),
    p = pchisq(d2, length(truth), lower.tail = FALSE)
  )
}

print_check <- function(rc) {
  cat(sprintf(
    paste(
      "Covered by the 95%% interval: %d of %d. Largest |z|: %.2f.",
      "d^2 = %.1f on %d df, p = %.2f.\n"
    ),
    sum(rc$table$covers),
    rc$m,
    max(abs(rc$table$z)),
    rc$d2,
    rc$m,
    rc$p
  ))
}

# Seconds in each stage of the fit, and the calls of logpost and grad
print_timing <- function(fit) {
  t <- fit$timing
  cat(sprintf(
    paste0(
      "Time: mode and Hessian %.1f s, VB %.1f s, marginals %.1f s,\n",
      "total %.1f s. Calls: %d of logpost, %d of grad.\n"
    ),
    t[["laplace"]],
    t[["vb"]],
    t[["marginals"]],
    t[["total"]],
    fit$counts[["logpost"]],
    fit$counts[["grad"]]
  ))
}

plot_z <- function(rc, main) {
```

```
  z <- rc$table$z
  m <- length(z)
  par(mar = c(5.5, 4, 2, 1))
  plot(
    seq_len(m),
    z,
    pch = 19,
    col = ifelse(rc$table$covers, "grey20", "firebrick"),
    ylim = range(c(-3, 3, z)),
    xaxt = "n",
    xlab = "",
    ylab = expression(z[j]),
    main = main
  )
  abline(h = c(-1.96, 0, 1.96), lty = c(2, 1, 2), col = "grey60")
  labels <- rownames(rc$table)
  at <- if (m > 40) seq(1, m, by = 3) else seq_len(m)
  axis(1, at = at, labels = labels[at], las = 2, cex.axis = 0.6)
}
```

1 Confirmatory M2PL (Chapter 2)

1.1 The model

Respondent $i = 1, \dots, N$ answers J binary items and has K correlated latent factors,

$$P(y_{ij} = 1 \mid \xi_i) = \text{logistic}\left(d_j + \sum_{k=1}^{K} a_{jk}\xi_{ik}\right), \qquad \xi_i \sim N_K(0, R),$$

where $a_{jk} = 0$ unless the indicator matrix says that item j measures factor k, and R is a correlation matrix (the thesis's Section 2.2.1.2). The marginal likelihood of respondent i is a K-dimensional integral,

$$p(y_i \mid a, d, R) = \int \prod_{j=1}^{J} P(y_{ij} \mid \xi)^{y_{ij}} \{1 - P(y_{ij} \mid \xi)\}^{1-y_{ij}} \, \phi_K(\xi; 0, R) \, d\xi,$$

computed here by a product Gauss–Hermite rule with 15 nodes per factor, on the distinct response patterns. With $K = 2$, R has one correlation ρ. The priors are $a_{jk}, d_j \sim N(0, 3^2)$ and $\rho \sim U(-1, 1)$, and `inla_vanilla()` works on $(a, d, \operatorname{atanh}\rho)$, with an analytic gradient.

1.2 The hardest instance

The thesis simulates the M2PL with $(K, J) = (5, 50)$ and $(10, 200)$ at $N = 10{,}000$ (Section 2.4.1.2), and at $N = 2{,}000$ (Appendix 6.5). Those need 5- or 10-dimensional integrals per respondent and 150 or 650 parameters, beyond a product rule inside every call of `logpost`. The one condition within reach is Appendix 6.6: $K = 2$, $J = 15$, $N = 2{,}000$, $d_j \sim U(-1, 1)$, nonzero $a_{jk} \sim U(0.5, 1.5)$ and

$\rho = 0.5$. The thesis does not print its 15×2 indicator matrix, so 12 items load on one factor each, alternating, and 3 load on both.

```
set.seed(2026)
a_true <- m2pl_q * matrix(runif(length(m2pl_q), 0.5, 1.5), nrow(m2pl_q))
d_true <- runif(nrow(m2pl_q), -1, 1)
m2pl_truth <- setNames(
  c(a_true[m2pl_free], d_true, atanh(0.5)),
  m2pl_names
)
m2pl_pat <- collapse_patterns(sim_m2pl(2000, a_true, d_true, 0.5))
c(patterns = nrow(m2pl_pat$y), parameters = length(m2pl_truth))
```

```
#>   patterns parameters
#>       1739         34
```

1.3 Fit and recovery

```
m2pl_init <- setNames(
  c(rep(1, length(m2pl_free)), numeric(nrow(m2pl_q)), 0),
  m2pl_names
)
m2pl_fit <- inla_vanilla(
  m2pl_logpost,
  m2pl_init,
  m2pl_grad,
  y = m2pl_pat$y,
  count = m2pl_pat$count,
  nodes = gh_grid(15, 2)
)
diagnostics(m2pl_fit)
```

```
#>              npar         converged        iterations          grad_inf
#>                34                 1                34          1.74e-03
#>       grad_inf_rel           grad_l2    mode_shift_max         hess_cond
#>           7.26e-03          2.76e-03          4.08e-04          9.47e+00
#>         vb_applied      vb_shift_max     vb_kld_global           kld_max
#>                  1            0.1526            0.0672            0.0116
#>           kld_mean          nmad_max         nmad_mean unscanned_mass_max
#>             0.0017            0.0053            0.0018          5.18e-04
#>        vb_fallback       vb_step_max      mean_gap_max     ridge_bend_max
#>                  0          2.04e-05            0.0068            0.0564
#>      n_heavy_tails
#>                  0
```

```
print_timing(m2pl_fit)
```

```
#> Time: mode and Hessian 3.1 s, VB 51.1 s, marginals 37.3 s,
#> total 93.8 s. Calls: 1274 of logpost, 6335 of grad.
```

```
set.seed(1)
m2pl_rc <- recovery_check(m2pl_fit, m2pl_truth)
print_check(m2pl_rc)
```

```
#> Covered by the 95% interval: 33 of 34. Largest |z|: 2.19. d^2 = 34.7 on 34 df, p = 0.44.
```

```
m2pl_err <- abs(m2pl_rc$table$mean - m2pl_rc$table$true)
m2pl_mae <- tapply(m2pl_err, substr(m2pl_names, 1, 1), mean)
plot_z(m2pl_rc, "M2PL, K = 2, J = 15, N = 2,000")
```

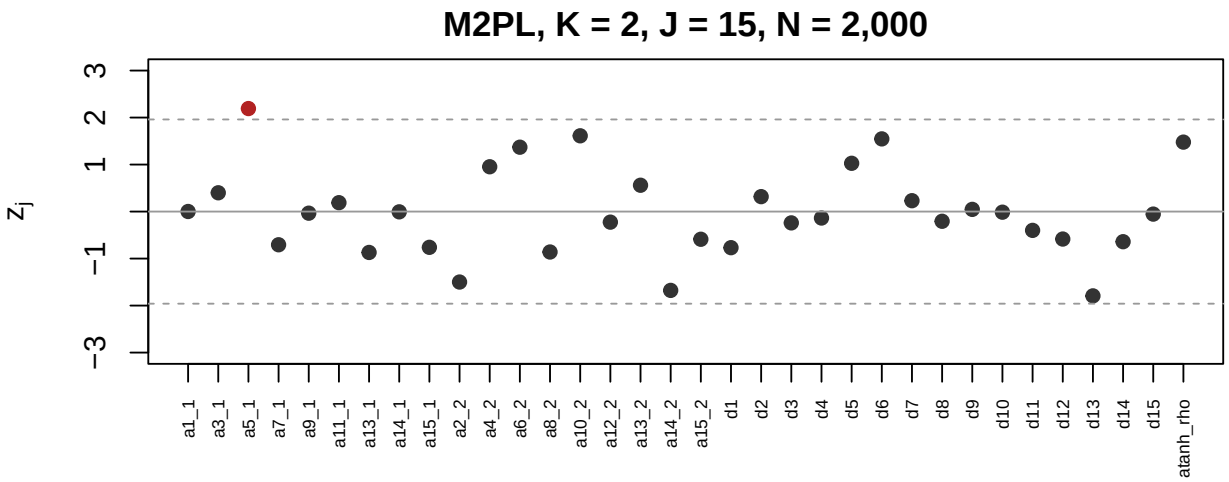


Figure 1: Standardised errors of the 34 parameters of the M2PL. Red: not covered by the 95% interval.

```
knitr::kable(m2pl_rc$table, digits = 3, booktabs = TRUE, longtable = TRUE)
```

	true	mean	sd	lower	upper	z	covers
a1_1	1.199	1.199	0.103	1.000	1.409	0.001	TRUE
a3_1	0.640	0.668	0.070	0.533	0.811	0.400	TRUE
a5_1	1.055	1.279	0.102	1.089	1.483	2.192	FALSE
a7_1	0.966	0.909	0.081	0.761	1.075	-0.706	TRUE
a9_1	0.753	0.750	0.074	0.612	0.904	-0.034	TRUE

	true	mean	sd	lower	upper	z	covers
a11_1	0.506	0.518	0.064	0.393	0.645	0.189	TRUE
a13_1	0.731	0.635	0.110	0.418	0.858	-0.870	TRUE
a14_1	1.349	1.348	0.162	1.055	1.690	-0.006	TRUE
a15_1	0.654	0.564	0.118	0.334	0.795	-0.761	TRUE
a2_2	1.045	0.918	0.085	0.755	1.090	-1.499	TRUE
a4_2	0.818	0.901	0.088	0.740	1.088	0.954	TRUE
a6_2	0.842	0.961	0.087	0.792	1.133	1.369	TRUE
a8_2	0.733	0.671	0.072	0.536	0.818	-0.860	TRUE
a10_2	0.912	1.060	0.091	0.886	1.247	1.612	TRUE
a12_2	0.911	0.892	0.083	0.733	1.062	-0.225	TRUE
a13_2	0.640	0.704	0.115	0.480	0.932	0.560	TRUE
a14_2	0.505	0.301	0.122	0.052	0.539	-1.677	TRUE
a15_2	1.102	1.022	0.136	0.762	1.298	-0.589	TRUE
d1	0.822	0.772	0.065	0.649	0.901	-0.768	TRUE
d2	-0.649	-0.631	0.057	-0.745	-0.520	0.318	TRUE
d3	0.462	0.450	0.051	0.352	0.549	-0.241	TRUE
d4	-0.968	-0.976	0.061	-1.097	-0.858	-0.137	TRUE
d5	0.172	0.234	0.060	0.115	0.353	1.027	TRUE
d6	-0.148	-0.065	0.054	-0.172	0.039	1.547	TRUE
d7	-0.579	-0.566	0.056	-0.676	-0.460	0.232	TRUE
d8	-0.046	-0.056	0.050	-0.152	0.040	-0.206	TRUE
d9	0.510	0.512	0.051	0.411	0.614	0.046	TRUE
d10	-0.106	-0.107	0.056	-0.216	0.002	-0.013	TRUE
d11	-0.258	-0.277	0.049	-0.371	-0.182	-0.400	TRUE
d12	0.750	0.716	0.057	0.608	0.831	-0.584	TRUE
d13	-0.131	-0.235	0.058	-0.354	-0.122	-1.794	TRUE
d14	-0.120	-0.161	0.064	-0.288	-0.036	-0.641	TRUE
d15	-0.836	-0.840	0.070	-0.985	-0.708	-0.054	TRUE
atanh_rho	0.549	0.638	0.060	0.525	0.759	1.478	TRUE

The fit took 94 seconds, and every check of `inla_vanilla()` passes: the largest VB shift is 0.15 posterior SDs, and no marginal has heavy tails or a ridge bend over 0.25. 33 of the 34 true values lie in their 95% intervals, about what 95% coverage gives, the largest $|z_j|$ is 2.19, and $d^2 = 34.7$ on 34 degrees of freedom ($p = 0.44$). The mean absolute error of the posterior means is 0.079 for the loadings and 0.031 for the intercepts, against 0.082 and 0.058 for the thesis's QN-SOMALA at $N = 2{,}000$, $K = 5$ and $J = 50$ (Table 6.11).

The approximation itself was checked against Stan, with NUTS on the full joint and the 4,000 factor scores sampled (4 chains of 2,000 draws, $\widehat{R} \leq 1.003$, no divergences): every posterior mean agreed within 0.014 posterior SDs, every SD within 4%, and every 2.5% and 97.5% quantile within 0.11 SDs.

2 Multilevel logistic regression (Chapter 2)

2.1 The model

Cluster $i = 1, \ldots, N$ has J_i binary responses, and every covariate has a random coefficient (Section 2.2.1.1),

$$P(y_{ij} = 1 \mid \xi_i) = \text{logistic}(x_{ij}^\top \xi_i), \qquad \xi_i \sim N_K(\mu, \Sigma), \qquad x_{ij1} = 1,$$

so ξ_{i1} is the random intercept, $\xi_{i2}, \ldots, \xi_{iK}$ are random slopes, μ holds the fixed effects and Σ their covariance. Each cluster's ξ_i is integrated out by a product Gauss–Hermite rule on $\xi = \mu + \text{diag}(\sigma) L z$, with L the Cholesky factor of the correlation matrix of Σ. The priors are $\mu_k \sim N(0, 5^2)$, $\sigma_k \sim N^+(0, 1)$ and an LKJ(1) prior, uniform over correlation matrices, on the correlations, which enter through their canonical partial correlations on the atanh scale.

2.2 The hardest instance

The thesis has $K = 5$ ($J_i = 10$) and $K = 10$ ($J_i = 20$) random coefficients for $N = 10{,}000$ clusters (Section 2.4.1.1). The integral runs over all 10,000 clusters in every call of `logpost`, and a product rule of n nodes per coefficient costs n^K passes over the 100,000 responses. $K = 3$, with $6^3 = 216$ nodes, takes about a third of a second a call, and $K = 5$ would take minutes. So this keeps the thesis's $N = 10{,}000$, $J_i = 10$ and true values, $\mu_1 = 0.3$, slopes $\mu_k \sim U(0.1, 1.1)$, variances 0.1, covariances 0.05 and covariates $N(0, R)$ with correlations 0.25, and takes $K = 3$: nine parameters.

```
set.seed(2027)
mlr_mu <- c(0.3, runif(mlr_k - 1, 0.1, 1.1))
mlr_sigma <- matrix(0.05, mlr_k, mlr_k)
diag(mlr_sigma) <- 0.1
mlr_truth <- setNames(
  c(mlr_mu, log(sqrt(diag(mlr_sigma))), corr_to_w(cov2cor(mlr_sigma))),
  mlr_names
)
mlr_dat <- sim_mlr(10000, mlr_mu, mlr_sigma)
mean(mlr_dat$y)
```

```
#> [1] 0.5645
```

2.3 Fit and recovery

```
mlr_init <- setNames(
  c(runif(mlr_k, 0, 1.5), numeric(mlr_k + choose(mlr_k, 2))),
  mlr_names
)
mlr_fit <- inla_vanilla(
  mlr_logpost,
  mlr_init,
```

```
  y = mlr_dat$y,
  x = mlr_dat$x,
  nodes = gh_grid(mlr_n_gh, mlr_k)
)
diagnostics(mlr_fit)
```

```
#>               npar          converged         iterations           grad_inf
#>                  9                  1                 78           2.97e-03
#>       grad_inf_rel            grad_l2     mode_shift_max          hess_cond
#>           9.82e-03           4.78e-03           2.58e-05           4.98e+00
#>         vb_applied      vb_shift_max      vb_kld_global             kld_max
#>                  1             0.4338             0.1681              0.0941
#>           kld_mean           nmad_max          nmad_mean unscanned_mass_max
#>             0.0279             0.0251             0.0072           8.17e-03
#>        vb_fallback       vb_step_max       mean_gap_max     ridge_bend_max
#>                  0           6.37e-04             0.1018             0.1390
#>      n_heavy_tails
#>                  0
```

```
print_timing(mlr_fit)
```

```
#> Time: mode and Hessian 354.0 s, VB 104.6 s, marginals 176.5 s,
#> total 635.6 s. Calls: 1980 of logpost, 0 of grad.
```

```
set.seed(1)
mlr_rc <- recovery_check(mlr_fit, mlr_truth)
print_check(mlr_rc)
```

```
#> Covered by the 95% interval: 8 of 9. Largest |z|: 2.07. d^2 = 11.4 on 9 df, p = 0.25.
```

```
plot_z(mlr_rc, "Multilevel logistic regression, K = 3, N = 10,000")
```

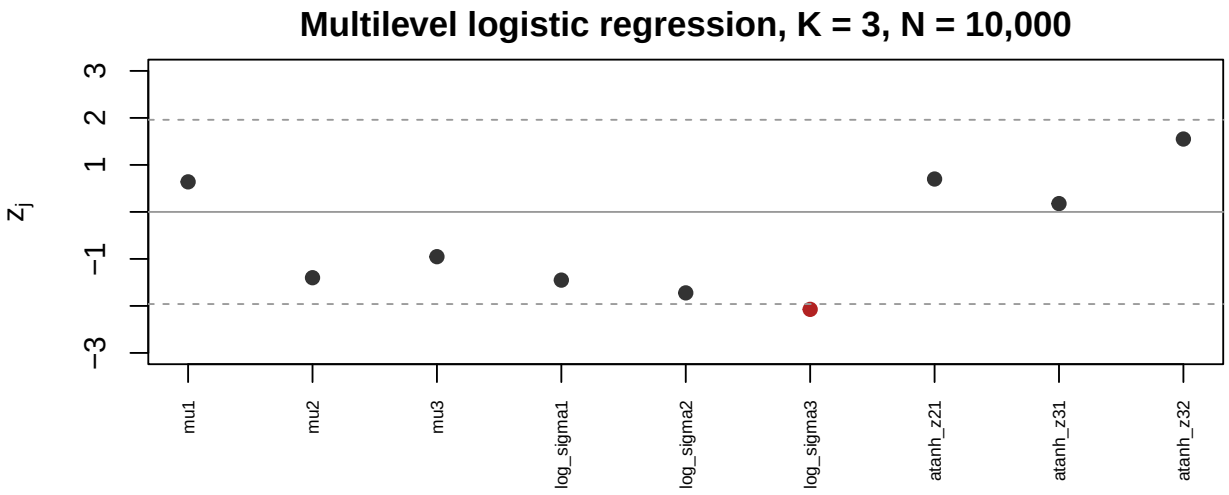


Figure 2: Standardised errors of the nine parameters of the multilevel logistic regression.

```
knitr::kable(mlr_rc$table, digits = 3, booktabs = TRUE)
```

	true	mean	sd	lower	upper	z	covers
mu1	0.300	0.305	0.008	0.289	0.321	0.640	TRUE
mu2	0.275	0.264	0.008	0.248	0.279	-1.400	TRUE
mu3	0.500	0.492	0.008	0.475	0.508	-0.952	TRUE
log_sigma1	-1.151	-1.222	0.049	-1.318	-1.128	-1.451	TRUE
log_sigma2	-1.151	-1.250	0.057	-1.367	-1.141	-1.722	TRUE
log_sigma3	-1.151	-1.281	0.063	-1.416	-1.167	-2.073	FALSE
atanh_z21	0.549	0.626	0.110	0.431	0.860	0.700	TRUE
atanh_z31	0.549	0.568	0.108	0.380	0.807	0.177	TRUE
atanh_z32	0.347	0.671	0.209	0.334	1.132	1.551	TRUE

On the natural scale, μ and the entries of Σ:

```
mlr_draws <- sampling(mlr_fit, nsamp = nsamp, fun = mlr_natural)
mlr_nat <- data.frame(
  true = mlr_natural(mlr_truth),
  mean = colMeans(mlr_draws),
  lower = apply(mlr_draws, 2, quantile, 0.025),
  upper = apply(mlr_draws, 2, quantile, 0.975)
)
mlr_nat$covers <- mlr_nat$lower <= mlr_nat$true & mlr_nat$true <= mlr_nat$upper
knitr::kable(mlr_nat, digits = 4, booktabs = TRUE)
```

	true	mean	lower	upper	covers
mu1	0.3000	0.3049	0.2896	0.3200	TRUE
mu2	0.2749	0.2638	0.2479	0.2806	TRUE
mu3	0.4995	0.4915	0.4741	0.5080	TRUE
sigma11	0.1000	0.0873	0.0710	0.1050	TRUE
sigma21	0.0500	0.0465	0.0346	0.0593	TRUE
sigma31	0.0500	0.0415	0.0295	0.0538	TRUE
sigma22	0.1000	0.0823	0.0640	0.1013	TRUE
sigma32	0.0500	0.0543	0.0417	0.0666	TRUE
sigma33	0.1000	0.0774	0.0590	0.0962	FALSE

```
mlr_err <- abs(mlr_nat$mean - mlr_nat$true)
is_mu <- startsWith(rownames(mlr_nat), "mu")
mlr_avg_mae <- (mean(mlr_err[is_mu]) + mean(mlr_err[!is_mu])) / 2
```

The fit took 10.6 minutes, nearly all of it in the 10,000 cluster integrals of each call of `logpost`. Every check passes but one borderline mean gap, of 0.1 posterior SDs, on the third canonical partial correlation. 8 of the 9 true values lie in their 95% intervals, the largest $|z_j|$ is 2.07, and $d^2 = 11.4$ on 9 degrees of freedom ($p = 0.25$). The thesis's error measure, the mean of the mean absolute errors of μ and of Σ, is 0.0098 here, against 0.0113 for its D-SOMALA at $K = 5$ (Table 2.2).

3 Grade of membership model (Chapter 3)

3.1 The model

Respondent i answers J binary items and belongs in part to each of K latent profiles, with a membership vector on the simplex (Section 3.2.1.1),

$$P(y_{ij} = 1 \mid \xi_i) = \sum_{k=1}^{K} \xi_{ik}\, \theta_{jk}, \qquad \xi_i \sim \text{Dirichlet}(\alpha),$$

where θ_{jk} is the probability that profile k answers item j. The thesis integrates ξ_i by Langevin MCMC on the sphere. But each factor $\sum_k \xi_{ik}\theta_{jk}^{y_{ij}}(1-\theta_{jk})^{1-y_{ij}}$ is linear in ξ_i, so the integrand is a polynomial of degree J, and the integral is exact by a Gauss rule. With $K = 3$, stick-breaking writes the Dirichlet as two independent betas, $\xi_1 = u$, $\xi_2 = (1-u)v$, $\xi_3 = (1-u)(1-v)$, with $u \sim \text{Beta}(\alpha_1, \alpha_2 + \alpha_3)$ and $v \sim \text{Beta}(\alpha_2, \alpha_3)$, and a Gauss–Jacobi rule with $\lceil (J+1)/2 \rceil$ nodes for each is exact.

The thesis fits by maximum likelihood. For a Bayesian fit the prior on θ_{jk} matters here: the data cannot tell a probability of 10^{-9} from one of 0.01, and about 40% of the true θ_{jk} lie within 0.01 of 0 or 1. Under a uniform prior those probabilities are pulled inwards and α drops to make up for it. In a Stan check of the data set below, even the exact posterior then missed all three true α. So the prior is the thesis's own generating distribution, $\theta_{jk} \sim \text{Beta}(0.2, 0.2)$, with $\log \alpha_k \sim N(0, 1)$. `inla_vanilla()` works on $\Phi^{-1}(\theta_{jk})$ and $\log \alpha_k$ ($m = 63$), from values of `logpost` alone. On the probit scale the prior's tail near 0 or 1 is normal, like that of $N(0, 5)$, where on the logit scale it would be exponential.

3.2 The hardest instance

The thesis has $K \in \{3, 6\}$, $J \in \{20, 40\}$ for $K = 3$ and $J \in \{40, 80\}$ for $K = 6$, uniform and non-uniform α, $N = 2{,}000$, and $\theta_{jk} \sim \text{Beta}(0.2, 0.2)$ (Section 3.3.1.1). With $K = 6$ the model has 246 or 486 parameters, and with $J = 40$ it has 123: too many for a fit that takes logpost values only. So this takes $K = 3$, $J = 20$ ($m = 63$) with the non-uniform $\alpha = (0.5, 1, 1.5)$, whose small first component puts many respondents near a vertex of the simplex.

```
set.seed(2028)
gom_theta <- matrix(rbeta(gom_j * gom_k, 0.2, 0.2), gom_j, gom_k)
gom_pat <- collapse_patterns(sim_gom(2000, gom_theta, gom_alpha))
mean(gom_theta < 0.01 | gom_theta > 0.99) # share of theta near 0 or 1
```

```
#> [1] 0.4333333
```

3.3 Fit and recovery

```
gom_init <- setNames(
  c(qnorm(rep(c(0.4, 0.5, 0.6), each = gom_j)), rep(log(2), 3)),
  gom_names
)
gom_fit <- inla_vanilla(
  gom_logpost,
  gom_init,
  y = gom_pat$y,
  count = gom_pat$count
)
diagnostics(gom_fit)
```

```
#>               npar          converged         iterations           grad_inf
#>                 63                  1                137           3.02e-03
#>       grad_inf_rel           grad_l2     mode_shift_max          hess_cond
#>           1.29e-01          6.41e-03          4.57e-04           2.85e+02
#>          vb_applied      vb_shift_max      vb_kld_global             kld_max
#>                   1            1.2215           10.8430              0.7461
#>            kld_mean          nmad_max         nmad_mean unscanned_mass_max
#>              0.3003            0.0908            0.0145           9.78e-01
#>         vb_fallback       vb_step_max       mean_gap_max     ridge_bend_max
#>                   1          8.16e-01           51.0536            21.7665
#>      n_heavy_tails
#>                  46
```

```
print_timing(gom_fit)
```

```
#> Time: mode and Hessian 62.9 s, VB 485.5 s, marginals 61.8 s,
#> total 610.2 s. Calls: 168099 of logpost, 0 of grad.
```

The profiles carry no labels, so they are matched to the true ones first.

```
perms <- as.matrix(expand.grid(1:3, 1:3, 1:3))
perms <- perms[apply(perms, 1, function(p) all(sort(p) == 1:3)), ]
est <- pnorm(matrix(gom_fit$summary[gom_theta_ix, "Mean"], gom_j))
gap <- apply(perms, 1, function(p) sum(abs(est - gom_theta[, p])))
perm <- perms[which.min(gap), ]
gom_truth <- setNames(
  c(qnorm(gom_theta[, perm]), log(gom_alpha[perm])),
  gom_names
)
set.seed(1)
gom_rc <- recovery_check(gom_fit, gom_truth)
print_check(gom_rc)
```

```
#> Covered by the 95% interval: 42 of 63. Largest |z|: 9.16. d^2 = 1627.0 on 63 df, p = 0.00.
```

```
plot_z(gom_rc, "Grade of membership, K = 3, J = 20, N = 2,000")
```

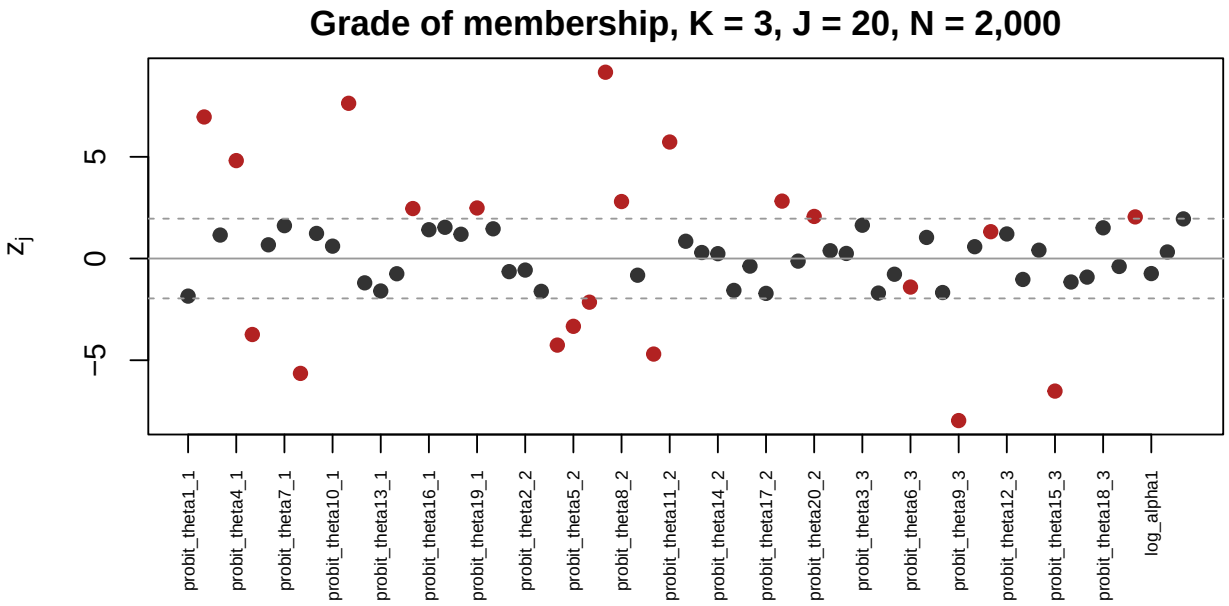


Figure 3: Standardised errors of the 63 parameters of the grade of membership model.

On the natural scale:

```r
gom_draws <- sampling(gom_fit, nsamp = nsamp, fun = function(theta) {
  c(pnorm(theta[gom_theta_ix]), exp(theta[gom_alpha_ix]))
})
post_mean <- colMeans(gom_draws)
gom_mae <- c(
  theta = mean(abs(post_mean[gom_theta_ix] - gom_theta[, perm])),
  alpha = mean(abs(post_mean[gom_alpha_ix] - gom_alpha[perm]))
)
gom_mae
```

```
#>      theta      alpha
#> 0.03058388 0.05662522
```

```r
gom_alpha_tab <- rbind(
  true = gom_alpha[perm],
  mean = post_mean[gom_alpha_ix],
  lower = apply(gom_draws[, gom_alpha_ix], 2, quantile, 0.025),
  upper = apply(gom_draws[, gom_alpha_ix], 2, quantile, 0.975)
)
colnames(gom_alpha_tab) <- paste0("alpha", seq_len(gom_k))
round(gom_alpha_tab, 3)
```

```
#>       alpha1 alpha2 alpha3
#> true   1.500  1.000  0.500
#> mean   1.428  1.022  0.576
#> lower  1.243  0.900  0.496
#> upper  1.615  1.147  0.658
```

The fit took 10.2 minutes, of which 8.1 went to the VB correction: it falls back to an optimiser and does not settle, and costs most of the 168,099 calls of `logpost`.

This is the one model where the approximation falls short. The point estimates recover the truth: the posterior means of α are 1.43, 1.02, 0.58 against 1.5, 1, 0.5, and the mean absolute errors are 0.031 for θ and 0.057 for α, against about 0.03 and 0.09 for the thesis's manifold MALA after 500 seconds (Figure 3.3). But only 42 of the 63 true values lie in their 95% intervals, and d^2 is far beyond χ^2_{63}. The misses are mostly the θ_{jk} near 0 or 1, whose posterior is a normal cut off by the likelihood, with a long tail: the skew-normal marginals are too narrow there, and for a few parameters wrong. `inla_vanilla()` says so. Its VB correction does not settle, and `summary()` flags heavy tails, mass beyond the scanned window and ridge bends for these parameters.

A Stan reference on the full joint, with the 2,000 membership vectors sampled, the same priors and 4 chains of 1,000 draws (each chain relabelled to the same profiles, $\hat{R} \leq 1.03$ after that, 80 divergent transitions), covered 62 of the 63 true values. So the posterior itself recovers the parameters, and it is the skew-normal approximation that does not.

4 Multilevel latent regression scaling model (Chapter 4)

4.1 The model

Student i in school g takes one booklet of items whose parameters are known from calibration: 2PL, 3PL and generalised partial credit (GPCM) items (Section 4.2.1). The scaling model regresses the trait on background predictors, with a random intercept per school,

$$\theta_{gi} = \eta_g + x_{gi}^{\top}\beta + e_{gi}, \qquad e_{gi} \sim N(0, \sigma^2), \qquad \eta_g \sim N(\mu, \tau^2),$$

and, for example, a 3PL item has $P(y_{gij} = 1 \mid \theta) = c_j + (1 - c_j)/[1 + \exp\{-a_j(\theta - b_j)\}]$. The thesis samples everything by MCMC. Here, since the items are fixed, each student's likelihood $L_{gi}(\theta)$ never changes, and its integral against $N(\eta_g + x_{gi}^{\top}\beta, \sigma^2)$ is a 30-node Gauss–Hermite rule placed once where L_{gi} has its mass. The school intercepts then integrate out by adaptive Gauss–Hermite quadrature, five nodes at each school's mode. The priors follow the thesis's families with values of our own: $\beta \mid s^2 \sim N(0, s^2 I)$ with $s^2 \sim \text{IG}(1, 1)$, which integrates to a multivariate t, $\sigma^2, \tau^2 \sim \text{IG}(1, 0.5)$ and $\mu \sim N(0, 10^2)$.

4.2 The hardest instance

The thesis's Simulation Study 1 (Section 4.4.1) has $N = 5{,}000$ students in $G = 50$ schools, 150 items in 5 booklets of 30 (30 GPCM, 60 3PL, 60 2PL), 60 predictors (20 continuous, 20 binary, 20 with three categories: 80 columns), 13 of them nonzero in the unidimensional condition, and $\tau^2 = 0.1$, $\sigma^2 = 1$. The bidimensional condition needs 2-dimensional integrals for every student and school and about 45 parameters, out of reach, so this is the unidimensional one, fitted with the 13 true predictors: the scaling model that a perfect selection step would refit. The thesis draws the item parameters between the smallest and largest TIMSS 2023 values, which it does not print: here $a \sim U(0.6, 2)$, $b \sim U(-2, 2)$, $c \sim U(0.05, 0.3)$.

```
set.seed(2029)
scal_dat <- sim_scaling(50, scal_per)
scal_rule <- scal_inner_rule(scal_dat, scal_dat$train | scal_dat$test, scal_r)
sel <- which(scal_dat$beta != 0)
scal_xs <- scal_dat$x[, sel, drop = FALSE]
scal_names <- c(sprintf("b_%s", colnames(scal_dat$x)[sel]), "mu", "log_sigma", "log_tau")
scal_truth <- setNames(c(scal_dat$beta[sel], 0, 0, log(sqrt(0.1))), scal_names)
length(scal_truth)
```

```
#> [1] 19
```

4.3 Fit and recovery

```
scal_init <- setNames(c(numeric(length(sel)), 0, 0, log(0.5)), scal_names)
scal_fit <- inla_vanilla(
  scal_logpost,
  scal_init,
  rule = scal_rule,
  xs = scal_xs,
  school = scal_dat$school
)
diagnostics(scal_fit)
```

```
#>               npar          converged         iterations           grad_inf
#>                 19                  1                  1           6.15e-02
#>       grad_inf_rel            grad_l2     mode_shift_max          hess_cond
#>           2.10e+00           6.67e-02           8.20e-04           6.66e+00
#>         vb_applied       vb_shift_max      vb_kld_global             kld_max
#>                  1             0.1973             0.0343              0.0195
#>           kld_mean           nmad_max          nmad_mean unscanned_mass_max
#>             0.0019             0.0033             0.0002           2.65e-04
#>        vb_fallback        vb_step_max       mean_gap_max     ridge_bend_max
#>                  0           2.37e-05             0.0036              0.0209
#>      n_heavy_tails
#>                  0
```

```
print_timing(scal_fit)
```

```
#> Time: mode and Hessian 59.5 s, VB 17.8 s, marginals 27.1 s,
#> total 104.5 s. Calls: 7395 of logpost, 0 of grad.
```

```
set.seed(1)
scal_rc <- recovery_check(scal_fit, scal_truth)
print_check(scal_rc)
```

```
#> Covered by the 95% interval: 18 of 19. Largest |z|: 2.09. d^2 = 16.6 on 19 df, p = 0.62.
```

```
plot_z(scal_rc, "Scaling model, K = 1, N = 5,000, G = 50")
```

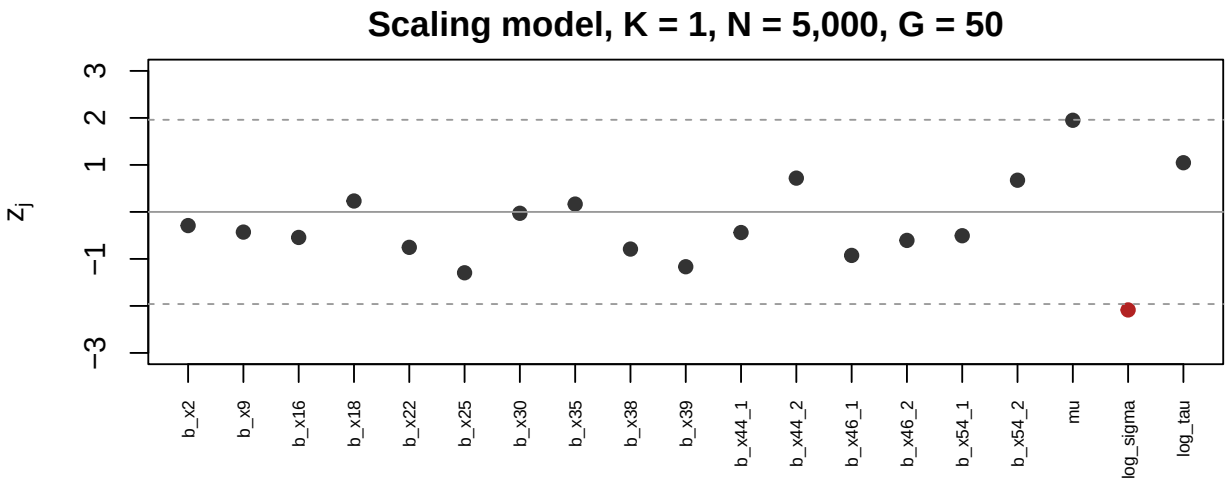


Figure 4: Standardised errors of the parameters of the scaling model.

```
knitr::kable(scal_rc$table, digits = 3, booktabs = TRUE)
```

	true	mean	sd	lower	upper	z	covers
b_x2	-0.493	-0.498	0.017	-0.532	-0.465	-0.290	TRUE
b_x9	0.381	0.374	0.017	0.340	0.407	-0.429	TRUE
b_x16	-0.339	-0.348	0.016	-0.379	-0.317	-0.543	TRUE
b_x18	-0.244	-0.240	0.016	-0.271	-0.208	0.232	TRUE
b_x22	-0.276	-0.300	0.032	-0.363	-0.234	-0.754	TRUE
b_x25	0.249	0.208	0.032	0.144	0.270	-1.295	TRUE
b_x30	-0.776	-0.777	0.032	-0.841	-0.712	-0.029	TRUE
b_x35	-0.355	-0.349	0.033	-0.413	-0.285	0.167	TRUE
b_x38	-0.577	-0.602	0.032	-0.663	-0.539	-0.790	TRUE
b_x39	-0.355	-0.393	0.032	-0.457	-0.329	-1.166	TRUE
b_x44_1	-0.560	-0.577	0.038	-0.651	-0.498	-0.439	TRUE
b_x44_2	-0.622	-0.592	0.042	-0.677	-0.510	0.719	TRUE
b_x46_1	-0.692	-0.728	0.039	-0.802	-0.651	-0.925	TRUE
b_x46_2	-0.228	-0.253	0.041	-0.335	-0.172	-0.606	TRUE
b_x54_1	0.602	0.583	0.038	0.509	0.658	-0.507	TRUE
b_x54_2	-0.226	-0.197	0.043	-0.279	-0.115	0.675	TRUE
mu	0.000	0.139	0.071	-0.001	0.280	1.950	TRUE
log_sigma	0.000	-0.029	0.014	-0.057	-0.001	-2.085	FALSE
log_tau	-1.151	-1.036	0.110	-1.242	-0.815	1.047	TRUE

The fit took 105 seconds, and every check passes. 18 of the 19 true values lie in their 95% intervals, the largest $|z_j|$ is 2.09, and $d^2 = 16.6$ on 19 degrees of freedom ($p = 0.62$). The quadrature is checked in `lab/oka-thesis.R`: each student's 30-node rule agrees with a fine grid to about 10^{-4}, and five adaptive nodes per school with fifteen to 10^{-7}. That script also runs the thesis's Bayesian

cross-validation on held-out items, which prefers the scaling model with the true predictors to one without predictors.

5 Summary

Three of the four models recover their parameters as well as one data set can show. For the M2PL, the multilevel logistic regression and the scaling model, `inla_vanilla()`'s checks pass (but for one borderline mean gap in the second), 33 of 34, 8 of 9 and 18 of 19 true values lie in their 95% intervals, and d^2 is unremarkable against χ^2_m. For the M2PL, a Stan reference confirms the marginals themselves. The grade of membership model is the exception. Its point estimates recover the truth, but the marginals of the item probabilities near 0 or 1 are too narrow, and `inla_vanilla()` flags them.

The times, in seconds, with the number of parameters m and the calls of `logpost` and its gradient:

	m	logpost	grad	mode	VB	marginals	total
M2PL	34	1,274	6,335	3.1	51.1	37.3	93.8
Multilevel logistic	9	1,980	0	354.0	104.6	176.5	635.6
Grade of membership	63	168,099	0	62.9	485.5	61.8	610.2
Scaling model	19	7,395	0	59.5	17.8	27.1	104.5

The time goes where each `logpost` call is dear or the calls are many. In the multilevel logistic regression every call integrates over 10,000 clusters (about a third of a second each), and in the grade of membership model the VB step that does not settle takes most of the calls. The thesis gives no time for these conditions. For scale, its QN-SOMALA took 44 seconds for the M2PL with $K = 5$, $J = 50$ and $N = 2{,}000$ (Table 6.11), and its simulations ran each algorithm for 500 to 2,000 seconds.

What sets the limit for all four is the integral inside `logpost`. The thesis's larger conditions, with more latent dimensions, more profiles or two traits, need integrals or numbers of parameters out of reach of a quadrature rule run thousands of times. That is the scale its stochastic optimisation was built for.

Reference

Oka, M. (2026). *Towards scalable psychometrics: Estimation, selection and evaluation of large-scale latent variable models* [Doctoral thesis, London School of Economics and Political Science].